0017. Letter Combinations of a Phone Number

egory: Backtracking | LeetCode: https://leetcode.com/problems/letter-combinatio

Problem

- Given digits 2-9, return every letter string the digits could represent on a phone keypad. Empty input → [].

Intuition

- One recursion level per digit; at each level, branch over that digit's 3-4 letters.

Brute Force

- Iterative cartesian product (BFS-style) is equally optimal; recursion reads more clearly.

Optimal Approach

- MAP = {"", "", "abc", "def", ...}.
- Dfs(i): if i == len record; for ch in MAP[digit i]: append, dfs(i+1), delete.

Worked Example

```
"23".

a -> ad ae af
b -> bd be bf
c -> cd ce cf
9 combinations
```

Java Solution

```java
public class LetterCombinationsOfPhoneNumber {
    private static final String[] MAP = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};

    public List<String> letterCombinations(String digits) {
        List<String> res = new ArrayList<>();
        if (digits.isEmpty()) return res;
        dfs(digits, 0, new StringBuilder(), res);
        return res;
    }

    private void dfs(String d, int i, StringBuilder sb, List<String> res) {
        if (i == d.length()) { res.add(sb.toString()); return; }
        for (char ch : MAP[d.charAt(i) - '0'].toCharArray()) {
            sb.append(ch);
            dfs(d, i + 1, sb, res);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

Complexity

- Time: O(4^n n). Space: O(n).

Pitfalls

- Empty input must return an empty list, not [""].